Universidad Distrital Francisco José de Caldas
Ingeniería Electrónica · Análisis y Diseño de Microprocesadores

TALLER 5

Microprocesador de propósito específico: determinante por el algoritmo de Bareiss con salida BCD (algoritmo XS3)

Explicación del diagrama ASM, del camino de datos y del código VHDL

Herramienta: Xilinx ISE 14.7 (XST, ISim) · Tarjeta: Digilent Nexys A7-100T (XC7A100T-1CSG324C)

Nota para el equipo. Este documento explica un diseño de referencia. El enunciado advierte que los diseños parecidos entre equipos no se califican, así que revisen cada estado, entiendan por qué está ahí y adáptenlo antes de entregarlo. Las conclusiones del informe son responsabilidad del equipo.

Contenido

1. Resumen del diseño **3**
Archivos del repositorio . . . 3
2. Repaso del algoritmo de Bareiss **3**
Ejemplo 3×3 (página 1 de la ROM) . . . 4
Pivote cero . . . 4
3. Algoritmo XS3 dentro del ASM **4**
Traza del XS3 para 362 (determinante de la página 3) . . . 4
4. Caja negra **6**
5. Cajas grises: unidad de control y camino de datos **6**
6. Pseudocódigo con el hardware disponible **7**
7. Diagrama ASM **8**
Convenciones . . . 8
Vista general . . . 8
Hoja 1: espera y carga de la matriz (S0 a S3) . . . 9
Hoja 2: pivote, búsqueda de fila e intercambio (S4 a S10) . . . 10
Hoja 3: productos y numerador (S11 a S16) . . . 11
Hoja 4: división por restas sucesivas y avance (S17 a S19) . . . 12
Hoja 5: determinante y valor a mostrar (S20 y S21) . . . 13
Hoja 6: algoritmo XS3 paso a paso y fin (S22 a S26) . . . 14
Tabla de estados . . . 15
8. Señales del procesador **16**
8.1 Señales de estado (CD → UC) . . . 16
8.2 Señales de control por estado (UC → CD) . . . 16
8.3 Registros internos y tamaños . . . 17
Por qué estos anchos . . . 17
9. Camino de datos **18**
10. Tiempo de ejecución **19**
11. Implementación en ISE 14.7 y la Nexys A7 **19**
11.1 Proyecto . . . 19
11.2 Operación en la tarjeta . . . 19
11.3 Matrices de prueba en la ROM . . . 19
11.4 Pines (src/nexys_a7.ucf) . . . 20
12. Simulación **20**
13. Limitaciones y verificación pendiente **21**
Anexo A. Código VHDL completo (src/det_bareiss.vhd) **22**

1. Resumen del diseño

El procesador lee de una ROM una matriz simétrica de 2×2 a 5×5, la copia a una RAM de trabajo y calcula su determinante con el algoritmo de Bareiss. Usa un multiplicador, un sumador/restador y una división por restas sucesivas. Guarda los pivotes en un banco de registros y convierte a BCD el valor que se va a mostrar con el algoritmo XS3 (sumar 3 y desplazar). La unidad de control es un ASM de 27 estados, S0 a S26, descrito en un solo proceso VHDL con el mismo estilo de los ejemplos de clase (`ASM_contador_unos` y `ASM_Sumax`): un `type estados_asm`, un `case st is` y solo señales.

Requisito del taller	Cómo lo cumple el diseño
ASM descrito en VHDL de alto nivel, con señal de inicio y señal de final	Un proceso con `case st is` (S0 a S26). `final = '1'` solo en S26. El cálculo arranca con `inicio` (BTNC).
Matriz en ROM: posición 00h = n, luego los a(i,j) como en la Figura 1	ROM de 256 × 6 bits con 8 páginas de 32 palabras. En cada página, 00h = n y los elementos van por columnas (a11, a21, …, an1, a12, …). `dira` de 5 bits; datos en complemento a 2 (−32 a 31).
Multiplicador, sumador/restador y división por restas sucesivas	Un multiplicador con signo de 25 × 25 bits, un sumador/restador de 50 bits compartido por todos los estados y una división por restas sucesivas (estados S16 y S17) que usa ese restador.
Memoria RAM para las iteraciones	RAM de 64 × 32 bits; la dirección es fila & columna (3 + 3 bits).
Diseño basado en señales	No hay variables en el diseño (el testbench sí usa variables).
Pivotes en un banco de registros, visibles con los switches	Banco de 4 × 32 bits. SW2..SW0 = 0 muestra el determinante; SW2..SW0 = k (1 a 4) muestra el pivote k.
Determinante en los displays	8 displays: AN7 = signo, AN6..AN0 = 7 dígitos. LD1 se enciende si el valor tiene más de 7 dígitos.
XS3 evidenciado paso a paso en el ASM	Estados S22 a S25: un estado por cada paso del documento XS3 (paso 1 = S22, paso 2 = S23, paso 3 = S24, paso 4 = S25, paso 5 = última pasada por S23).

Archivos del repositorio

Archivo	Contenido
src/det_bareiss.vhd	Diseño completo en un solo archivo: ROM, RAM, banco, unidades funcionales, ASM, XS3 y display.
src/nexys_a7.ucf	Pines y restricción de reloj (100 MHz) para ISE.
sim/tb_det_bareiss.vhd	Testbench: prueba las 8 matrices de la ROM leyendo el display y compara con los valores esperados.
sim/modelo_referencia.py	Modelo en Python: calcula det y pivotes esperados, cuenta ciclos y genera líneas de ROM.
sim/resultado_ghdl.txt	Salida de la simulación completa (GHDL).
doc/figuras/*.png	Diagramas ASM y de bloques en imagen.
doc/fuentes/*.py	Scripts que generan las figuras y este PDF.

2. Repaso del algoritmo de Bareiss

Bareiss es una eliminación gaussiana sin fracciones. En el paso k, cada elemento de la submatriz que está debajo y a la derecha del pivote se actualiza con la fórmula (1) del enunciado:

```
a(i,j) = ( a(k,k)·a(i,j) − a(i,k)·a(k,j) ) / p        para k < i ≤ n,  k < j ≤ n
p = pivote del paso anterior (p = 1 en el primer paso)
```

Después del paso k, el pivote actual a(k,k) pasa a ser p. Tras n − 1 pasos, el determinante queda en a(n,n). La división siempre es exacta: por la identidad de Sylvester, después del paso k cada a(i,j) es

el determinante de una submatriz de (k+1)×(k+1) de A, que es un entero. En la simulación el residuo R siempre termina en 0.

Durante el paso k solo cambian los elementos con i > k y j > k. La fila k y la columna k no se modifican, por eso el cálculo se hace sobre la misma RAM, sin una segunda copia de la matriz.

Ejemplo 3×3 (página 1 de la ROM)

```
A = | 2 1 3 |      k = 1, p = 1:  a22 = (2·4 − 1·1)/1 = 7    a23 = (2·2 − 1·3)/1 = 1
    | 1 4 2 |                      a32 = (2·2 − 3·1)/1 = 1    a33 = (2·5 − 3·3)/1 = 1
    | 3 2 5 |      k = 2, p = 2:  a33 = (7·1 − 1·1)/2 = 3   →   det = 3
Pivotes guardados en el banco: pivote(1) = 2, pivote(2) = 7
```

Pivote cero

Si a(k,k) = 0, el ASM busca la primera fila i > k con a(i,k) ≠ 0 (estado S5), intercambia las filas k e i en las columnas k a n (S6 a S9) y cambia el bit `signo`. Al final, det = −a(n,n) si hubo un número impar de intercambios. Si no hay ninguna fila con a(i,k) ≠ 0, la columna es cero debajo del pivote y det = 0. Las columnas menores que k no se intercambian porque el algoritmo no las vuelve a leer.

3. Algoritmo XS3 dentro del ASM

El documento XS3 describe 5 pasos. En el ASM cada paso es un estado propio, de modo que el diagrama muestra la conversión completa y no una caja que diga «convertir a BCD».

Paso del documento XS3	Estado	Operación en el hardware
1. Inicializar el registro decimal en ceros	S22	BCD ← 0 (10 dígitos); cont ← 0
2. Colocar el MSB del registro binario en el LSB del registro decimal	S23	BCD(0)(0) ← bin(31)
3. Revisar cada dígito: si es ≥ 5 (> 0100), sumarle 3 (0011)	S24	Un dígito por ciclo: d = 0 a 9. Si BCD(d) > 4, BCD(d) ← BCD(d) + 3 usando el sumador/restador.
4. Correr a la izquierda y descartar el bit que sale; repetir desde el paso 2 (bits − 1) veces	S25	BCD ← BCD << 1 (entra 0, que el paso 2 reemplaza), bin ← bin << 1, cont ← cont + 1; vuelve a S23.
5. Realizar únicamente el paso 2	S23 con cont = 31	En la pasada 32 por S23 se inserta el último bit y la decisión cont = 31 lleva a S26 sin pasar por S24 ni S25.

Por qué se suma 3: si un dígito vale 5 o más, al correrlo a la izquierda quedaría en 10 o más, que no es un dígito BCD válido. Sumar 3 antes del corrimiento equivale a sumar 6 después, que es la corrección BCD, y hace que el exceso pase como acarreo al dígito siguiente.

Tamaños: el registro binario `bin` tiene 32 bits, así que el registro decimal necesita 10 dígitos ($2^{32} - 1 = 4\,294\,967\,295$) y el lazo da 32 pasadas por el paso 2 (cont = 0 a 31). El signo no entra al XS3: el estado S21 convierte |valor| y guarda el signo en neg, que luego enciende el segmento g del display AN7.

Traza del XS3 para 362 (determinante de la página 3)

$362 = 1\,0110\,1010_2$ tiene 9 bits significativos. En las pasadas cont = 0 a 22 solo entran ceros y el registro decimal sigue en 0, por eso la tabla empieza en cont = 23. La columna «bin(31..23)» muestra los 9 bits de arriba del registro binario; el bit de la izquierda, bin(31), es el que copia el paso 2. El registro decimal se muestra con sus tres dígitos bajos: centenas, decenas y unidades.

cont	Estado (paso)	BCD: C D U	bin(31..23)	Comentario
23	S23 (2)	0000 0000 0001	101101010	

cont	Estado (paso)	BCD: C D U	bin(31..23)	Comentario
	S24 (3)	0000 0000 0001	101101010	ningún dígito > 4
	S25 (4)	0000 0000 0010	011010100	corrimiento a la izquierda
24	S23 (2)	0000 0000 0010	011010100	
	S24 (3)	0000 0000 0010	011010100	ningún dígito > 4
	S25 (4)	0000 0000 0100	110101000	corrimiento a la izquierda
25	S23 (2)	0000 0000 0101	110101000	
	S24 (3)	0000 0000 1000	110101000	BCD(0) = 0101 > 0100: +3 = 1000
	S25 (4)	0000 0001 0000	101010000	corrimiento a la izquierda
26	S23 (2)	0000 0001 0001	101010000	
	S24 (3)	0000 0001 0001	101010000	ningún dígito > 4
	S25 (4)	0000 0010 0010	010100000	corrimiento a la izquierda
27	S23 (2)	0000 0010 0010	010100000	
	S24 (3)	0000 0010 0010	010100000	ningún dígito > 4
	S25 (4)	0000 0100 0100	101000000	corrimiento a la izquierda
28	S23 (2)	0000 0100 0101	101000000	
	S24 (3)	0000 0100 1000	101000000	BCD(0) = 0101 > 0100: +3 = 1000
	S25 (4)	0000 1001 0000	010000000	corrimiento a la izquierda
29	S23 (2)	0000 1001 0000	010000000	
	S24 (3)	0000 1100 0000	010000000	BCD(1) = 1001 > 0100: +3 = 1100
	S25 (4)	0001 1000 0000	100000000	corrimiento a la izquierda
30	S23 (2)	0001 1000 0001	100000000	
	S24 (3)	0001 1011 0001	100000000	BCD(1) = 1000 > 0100: +3 = 1011
	S25 (4)	0011 0110 0010	000000000	corrimiento a la izquierda
31	S23 (2)	0011 0110 0010	000000000	último PASO 2 = PASO 5, va a S26

Resultado: BCD = 0011 0110 0010 = 3 6 2. La simulación del VHDL muestra « 0000362» en el display para esta matriz.

4. Caja negra

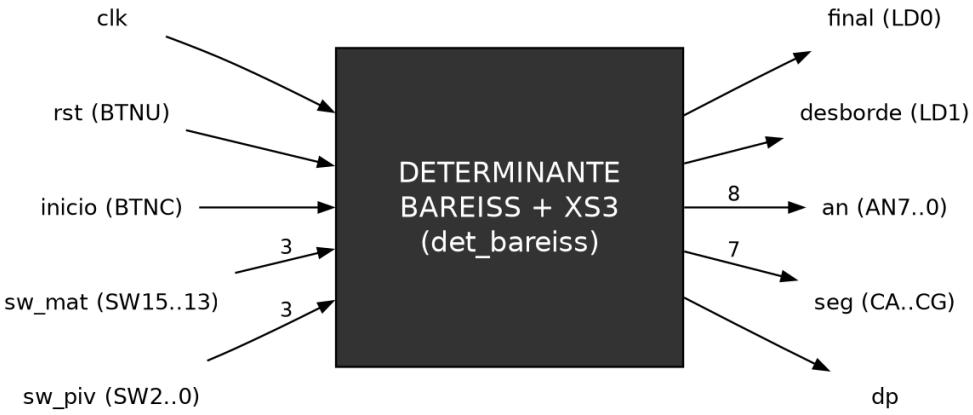


Figura 1. Caja negra del procesador (entidad det_bareiss).

Puerto	Dir.	Bits	Pin Nexys A7	Función
clk	in	1	E3	Reloj de 100 MHz
rst	in	1	BTNU (M18)	Reinicio asíncrono, activo en alto
inicio	in	1	BTNC (N17)	Comienza el cálculo
sw_mat	in	3	SW15..SW13	Elige la matriz de prueba (página de la ROM)
sw_piv	in	3	SW2..SW0	0: muestra el determinante; 1 a 4: muestra el pivote k
final	out	1	LD0	'1' cuando el valor ya está en el display (estado S26)
desborde	out	1	LD1	El valor mostrado tiene más de 7 dígitos
an	out	8	AN7..AN0	Ánodos del display (activos en bajo)
seg	out	7	CA..CG	Segmentos; seg(0) = CA … seg(6) = CG (activos en bajo)
dp	out	1	DP	Punto decimal, siempre apagado

5. Cajas grises: unidad de control y camino de datos

La unidad de control (UC) es el ASM: su registro de estado `st` y la lógica de estado siguiente. El camino de datos (CD) contiene memorias, registros y unidades funcionales. La UC le entrega al CD el estado actual; con él se forman las señales de control (escritura y dirección de la RAM, operandos del sumador/restador, carga de registros). El CD le devuelve a la UC las señales de estado que aparecen en las cajas de decisión (sección 8).

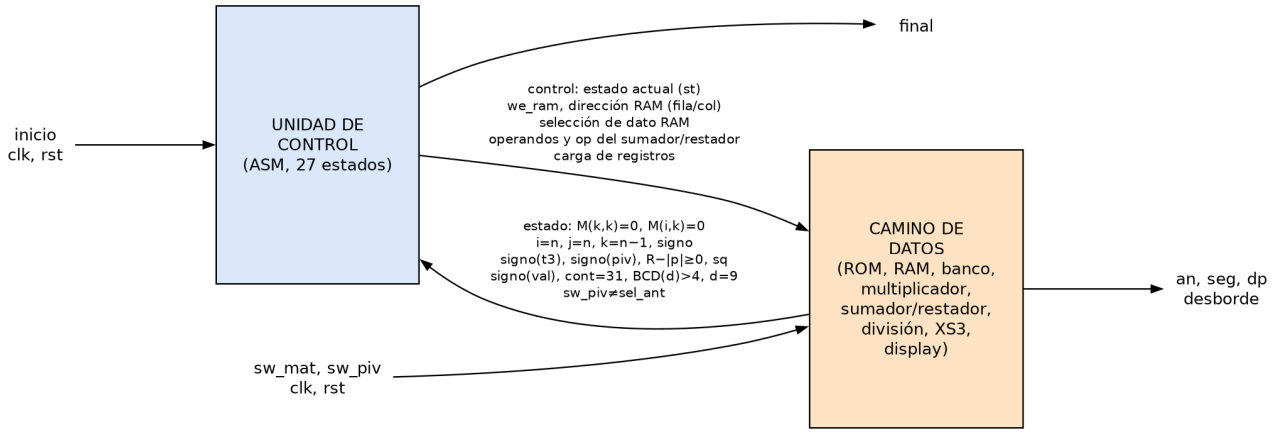


Figura 2. Cajas grises: señales de control (UC → CD) y señales de estado (CD → UC).

6. Pseudocódigo con el hardware disponible

Cada línea indica el estado del ASM que la ejecuta. Las operaciones del mismo estado ocurren en el mismo ciclo de reloj.

```
S0 :  final = 0;  esperar inicio = 1;  pag = sw_mat
S1 :  n = ROM(00h)
S2 :  para j = 1..n, i = 1..n:  M(i,j) = ROM(dira);  dira = dira + 1     (orden por columnas)
S3 :  k = 1;  p = 1;  |p| = 1;  signo = 0;  banco = 0
mientras k <= n-1:
  S4 :  piv = M(k,k)
        si piv = 0:
  S5 :     buscar i = k+1..n con M(i,k) /= 0
           si no existe:  det = 0;  ir a VISUALIZAR
  S6-S9:   intercambiar filas k e i (columnas k..n);  signo = not signo;  volver a S4
  S10:  banco(k) = piv
        para i = k+1..n, j = k+1..n:
  S11-S14:  t1 = piv · M(i,j);   t2 = M(i,k) · M(k,j)        (un multiplicador, dos usos)
  S15:      t3 = t1 - t2                                     (sumador/restador)
  S16-S17:  Q = |t3| / |p|  por restas sucesivas            (sumador/restador)
  S18:      M(i,j) = +Q  o  -Q   (signo = signo(t3) xor signo(p))
  S19:  p = piv;  |p| = |piv|;  k = k + 1
S20:  det = M(n,n);  si signo = 1 entonces det = -det
VISUALIZAR:
S21:  val = det si sw_piv = 0, si no banco(sw_piv);  bin = |val|;  neg = signo(val)
S22:  XS3 paso 1:  BCD = 0;  cont = 0
repetir:
  S23:  XS3 paso 2:  BCD(0)(0) = bin(31);   si cont = 31 salir   (paso 5)
  S24:  XS3 paso 3:  para d = 0..9:  si BCD(d) > 4 entonces BCD(d) = BCD(d) + 3
  S25:  XS3 paso 4:  BCD = BCD << 1;  bin = bin << 1;  cont = cont + 1
S26:  final = 1;  display = BCD;  si inicio = 1 ir a S1;  si cambia sw_piv ir a S21
```

7. Diagrama ASM

Convenciones

Elemento	Significado
Caja de estado (rectángulo azul)	Operaciones que se hacen en todo el estado (salidas tipo Moore).
Caja de decisión (rombo)	Pregunta sobre una señal de estado. La salida 1 es la condición verdadera.
Caja de salida condicional (rectángulo redondeado naranja)	Operación que solo ocurre si se toma ese camino (tipo Mealy), como en el ejemplo «ASM Mealy» de clase.
Bloque ASM (recuadro punteado)	Caja de estado más sus decisiones y salidas condicionales: dura un ciclo de reloj.
Conector (círculo verde)	Une hojas del diagrama: A → S1, B → S4, C → S11, D → S21, E → S20, F → S17, G → S22.
x ← expresión	Asignación a un registro; se hace efectiva al final del ciclo (flanco de subida).
x = expresión	Valor combinacional (no se guarda), por ejemplo final = 1 o la selección val.
M(i,j)	Palabra de la RAM en la dirección fila i, columna j.

Todas las decisiones de un bloque usan los valores que tienen los registros al comenzar el ciclo. Por ejemplo, en S19 la pregunta k = n − 1 usa el k anterior al incremento. El diagrama cumple las dos reglas vistas en clase: cada combinación de estado y decisiones lleva a un solo estado siguiente, y todo camino de decisiones termina en un estado.

Vista general

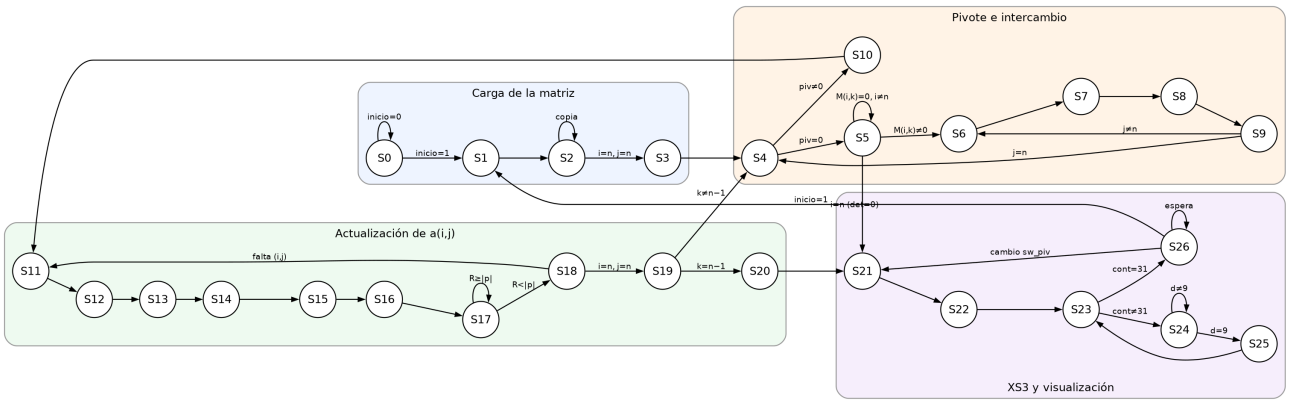


Figura 3. Diagrama de estados de la unidad de control (27 estados), agrupado por función.

Hoja 1: espera y carga de la matriz (S0 a S3)

S2 recorre la ROM con un solo contador de direcciones (`dira`) y dos contadores de fila y columna. La dirección de la RAM es fila & columna, así que no hace falta multiplicar para ubicar a(i,j).

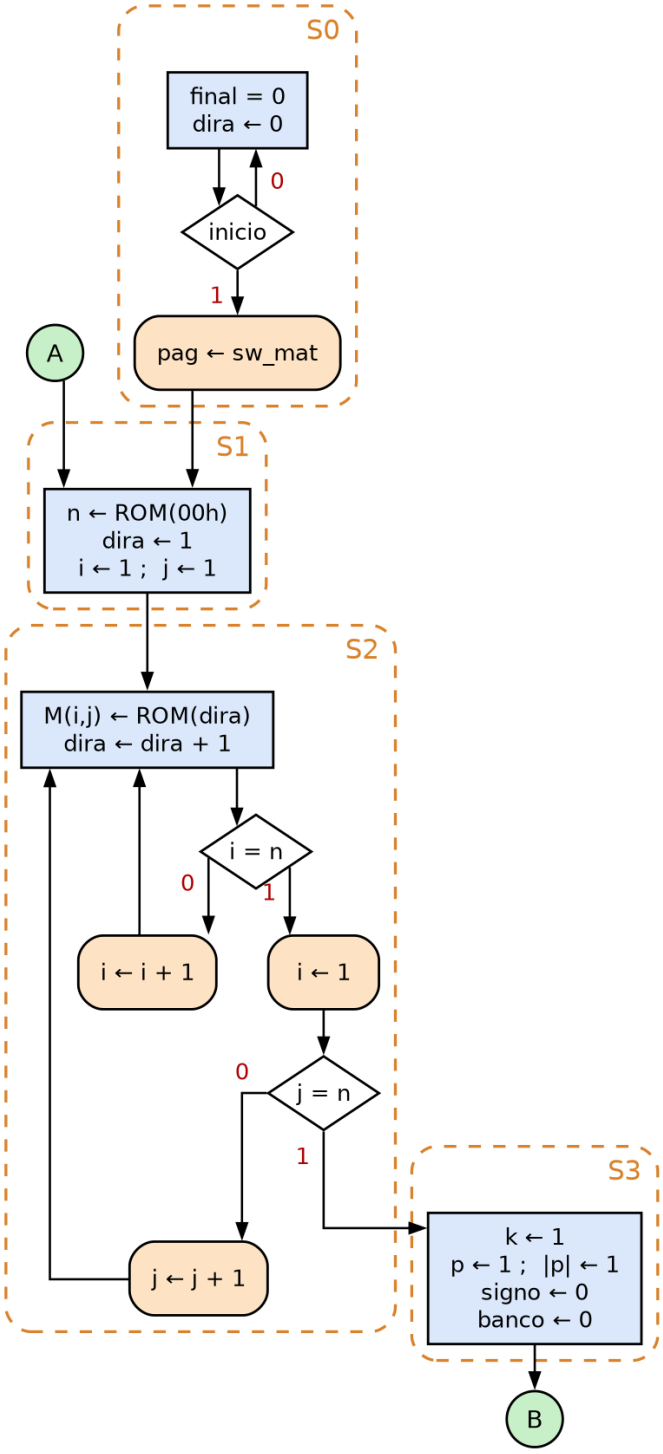


Figura 4. Hoja 1: espera y carga de la matriz (S0 a S3).

Hoja 2: pivote, búsqueda de fila e intercambio (S4 a S10)

La RAM tiene un solo puerto: cada lectura o escritura ocupa un estado, por eso el intercambio usa cuatro estados por columna. Después del intercambio se vuelve a S4 para leer el pivote nuevo, que ya no es cero.

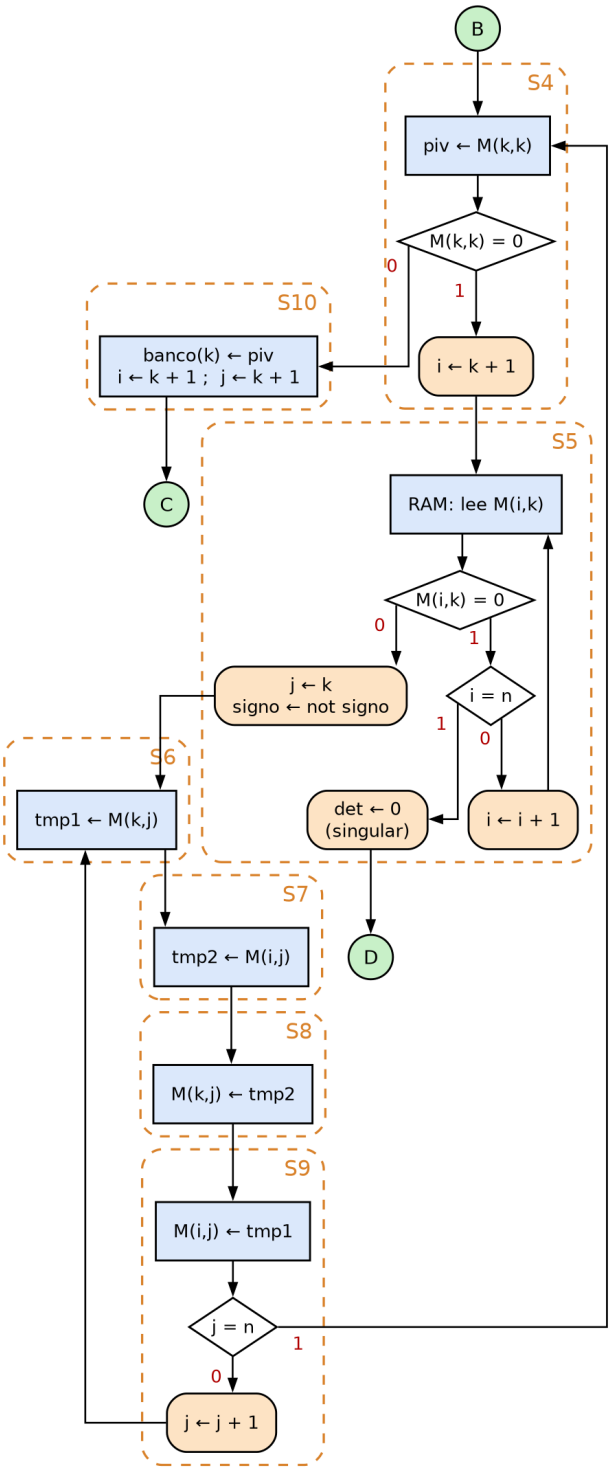


Figura 5. Hoja 2: pivote, búsqueda de fila e intercambio (S4 a S10).

Hoja 3: productos y numerador (S11 a S16)

Con un solo multiplicador, los dos productos de la fórmula (1) se hacen en estados distintos. Los operandos se cargan en los registros mul_a y mul_b un estado antes de usar el producto, para que el camino crítico sea registro → multiplicador → registro.

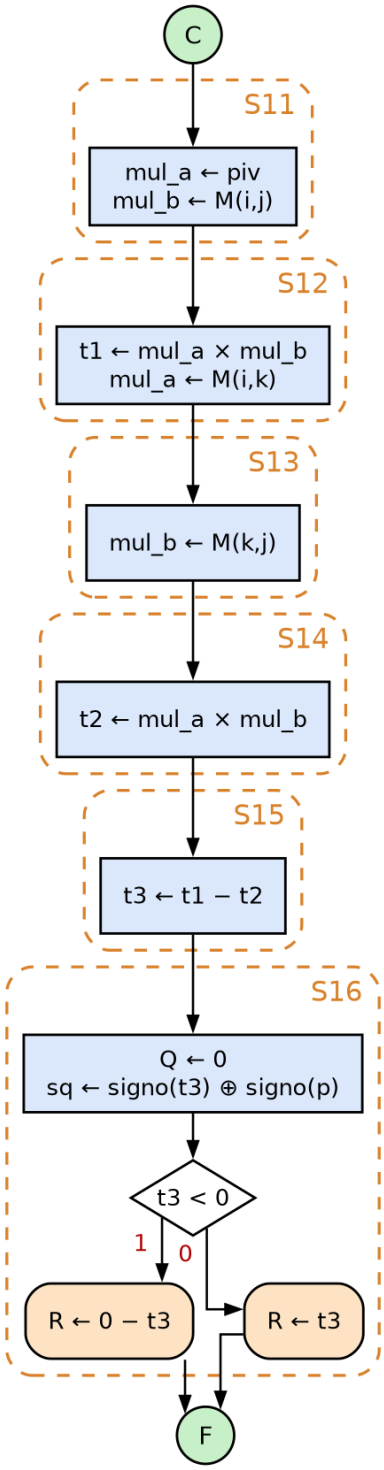


Figura 6. Hoja 3: productos y numerador (S11 a S16).

Hoja 4: división por restas sucesivas y avance (S17 a S19)

S17 es el bloque de división: en cada ciclo el restador calcula R − |p|; si el resultado no es negativo (bit 49 en 0) se guarda en R y el cociente Q aumenta en 1. La división se hace con magnitudes y el signo se aplica al escribir en S18.

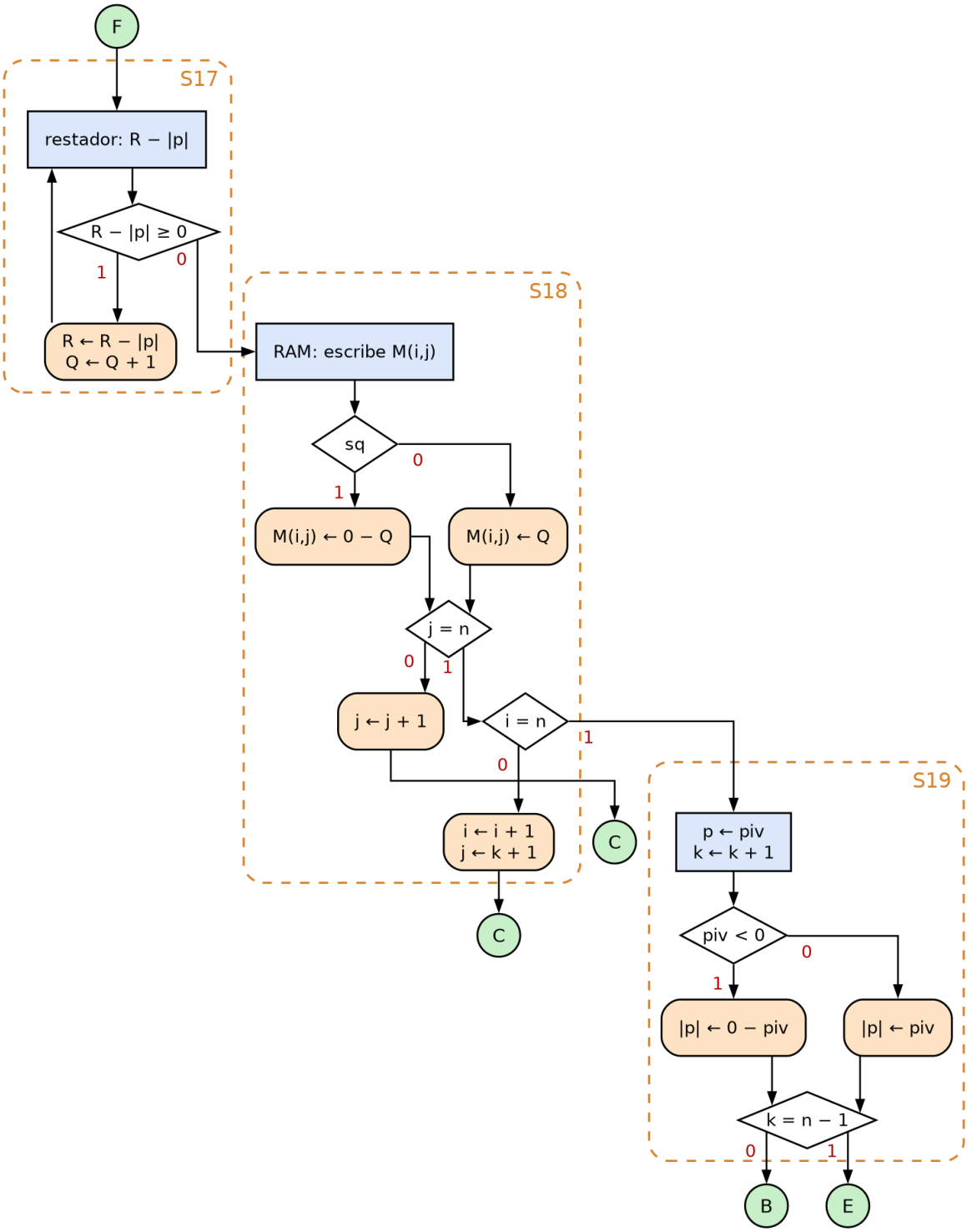


Figura 7. Hoja 4: división por restas sucesivas y avance (S17 a S19).

Hoja 5: determinante y valor a mostrar (S20 y S21)

S21 es la entrada a la conversión. Se llega desde S20 (cálculo normal), desde S5 (matriz singular, det = 0) y desde S26 cuando cambian los switches del banco de pivotes.

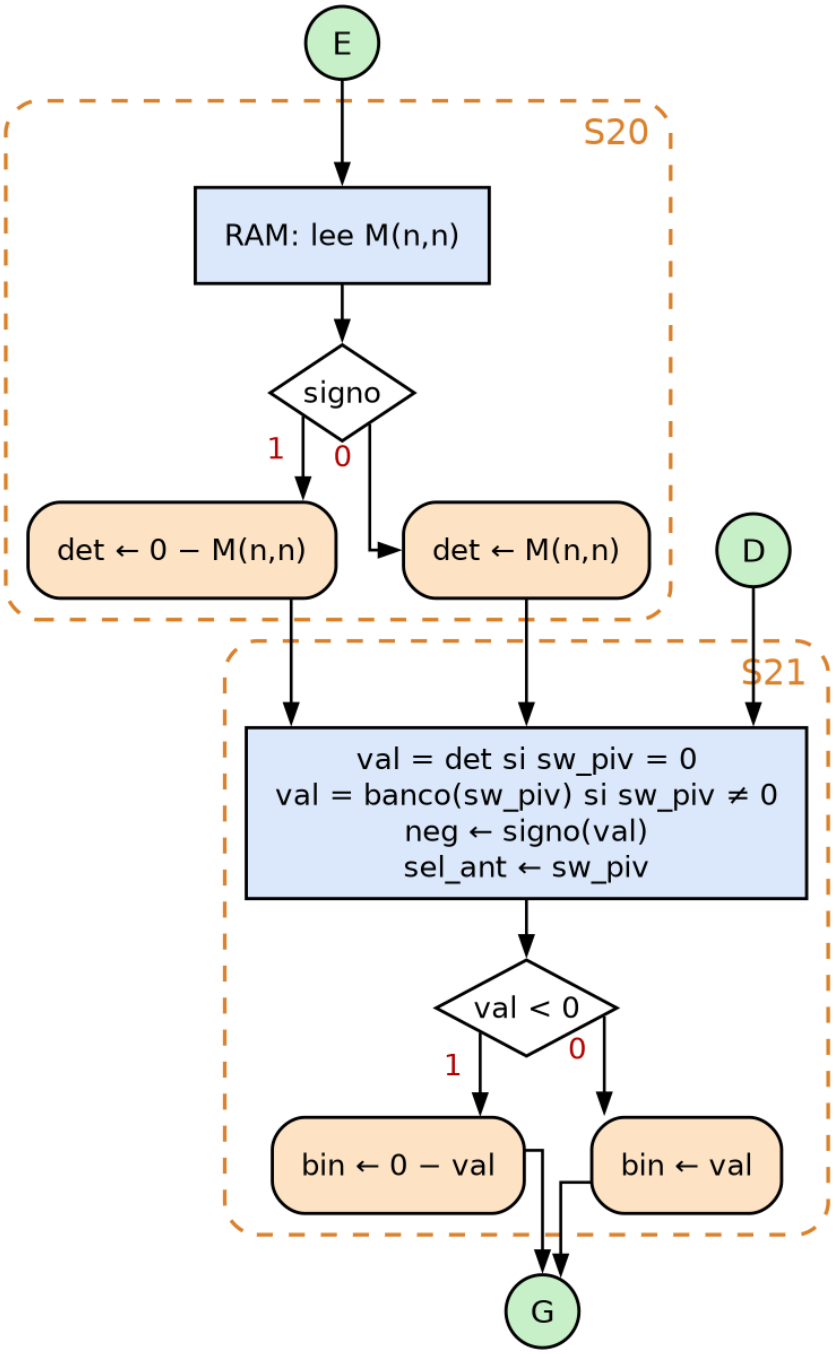


Figura 8. Hoja 5: determinante y valor a mostrar (S20 y S21).

Hoja 6: algoritmo XS3 paso a paso y fin (S22 a S26)

S26 se queda esperando con final = 1 y el valor en el display. Si el usuario mueve SW2..SW0, el ASM vuelve a S21 y repite el XS3 con el pivote elegido (unos 375 ciclos, invisibles para el ojo).

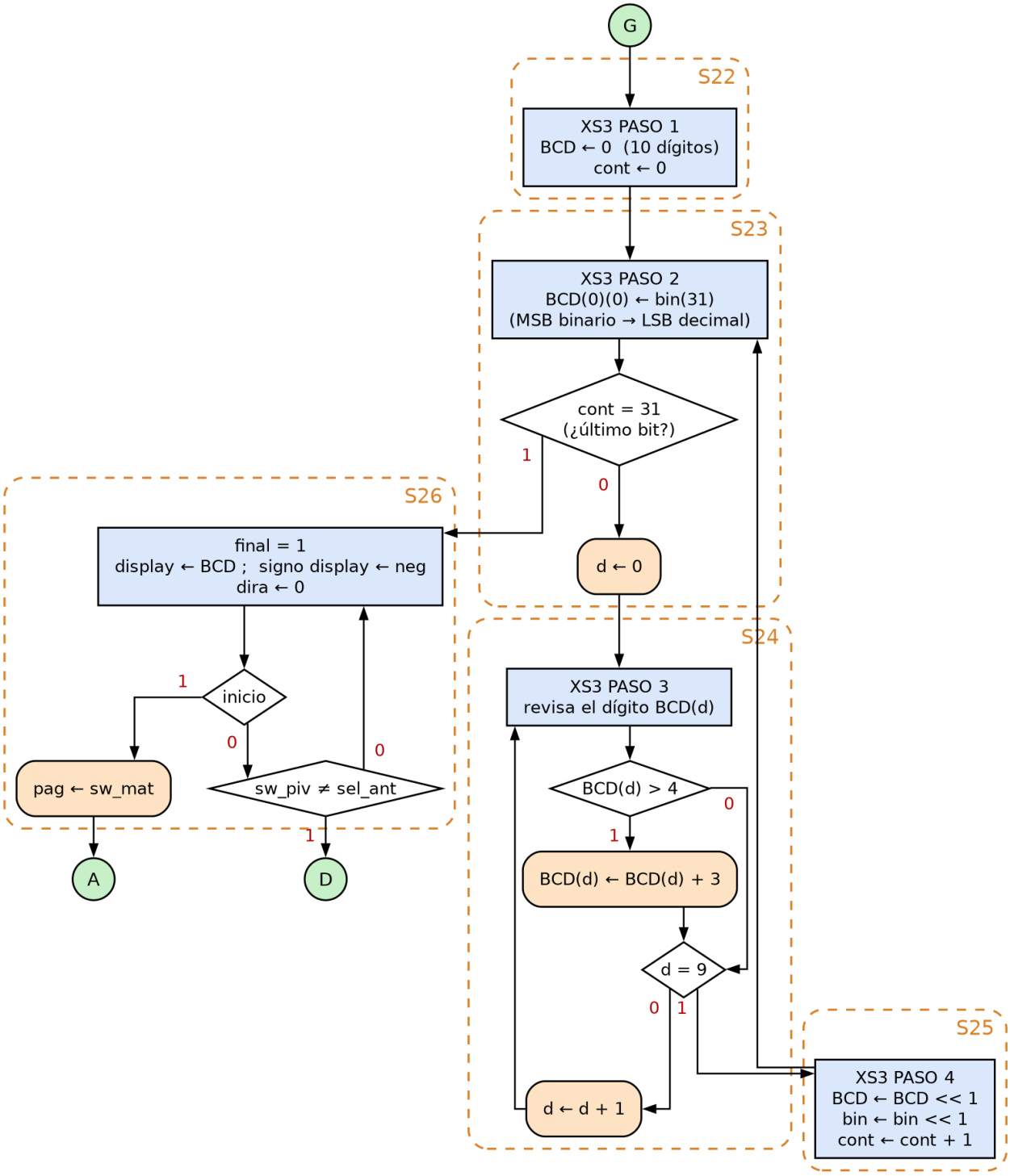


Figura 9. Hoja 6: algoritmo XS3 paso a paso y fin (S22 a S26).

Tabla de estados

Resumen de cada bloque ASM: lo que hace la caja de estado y las salidas condicionales, y a qué estado va según las decisiones. Es la misma información de las figuras 4 a 9, en el orden del código VHDL.

Estado	Operaciones	Siguiente estado
S0	final = 0; dira ← 0	inicio = 1: pag ← sw_mat, S1. Si no: S0.
S1	n ← ROM(00h); dira ← 1; i ← 1; j ← 1	S2
S2	M(i,j) ← ROM(dira) con extensión de signo a 32 bits; dira ← dira + 1. i es el índice que cambia más rápido, como en la Figura 1 del taller.	i ≠ n: i ← i + 1, S2. i = n y j ≠ n: i ← 1, j ← j + 1, S2. i = n y j = n: S3.
S3	k ← 1; p ← 1 (pivote inicial a(0,0) = 1); |p| ← 1; signo ← 0; banco ← 0	S4
S4	piv ← M(k,k)	M(k,k) = 0: i ← k + 1, S5. Si no: S10.
S5	Lee M(i,k) buscando una fila con elemento distinto de cero en la columna k.	M(i,k) ≠ 0: j ← k, signo ← not signo, S6. M(i,k) = 0 e i = n: det ← 0, S21 (singular). Si no: i ← i + 1, S5.
S6	tmp1 ← M(k,j)	S7
S7	tmp2 ← M(i,j)	S8
S8	M(k,j) ← tmp2	S9
S9	M(i,j) ← tmp1	j ≠ n: j ← j + 1, S6. j = n: S4 (vuelve a leer el pivote).
S10	banco(k) ← piv; i ← k + 1; j ← k + 1	S11
S11	mul_a ← piv; mul_b ← M(i,j)	S12
S12	t1 ← mul_a × mul_b (= a(k,k)·a(i,j)); mul_a ← M(i,k)	S13
S13	mul_b ← M(k,j)	S14
S14	t2 ← mul_a × mul_b (= a(i,k)·a(k,j))	S15
S15	t3 ← t1 − t2	S16
S16	Q ← 0; sq ← signo(t3) xor signo(p); R ← |t3| (0 − t3 si t3 < 0)	S17
S17	El restador calcula R − |p| (resta de prueba).	R − |p| ≥ 0: R ← R − |p|, Q ← Q + 1, S17. Si no: S18 (R queda en 0 porque la división es exacta).
S18	M(i,j) ← Q, o 0 − Q si sq = 1	j ≠ n: j ← j + 1, S11. j = n e i ≠ n: i ← i + 1, j ← k + 1, S11. i = n y j = n: S19.
S19	p ← piv; |p| ← |piv|; k ← k + 1	k = n − 1: S20. Si no: S4.
S20	det ← M(n,n), o 0 − M(n,n) si signo = 1	S21
S21	val = det si sw_piv = 0, banco(sw_piv) si no; neg ← signo(val); bin ← |val|; sel_ant ← sw_piv	S22
S22	XS3 paso 1: BCD ← 0; cont ← 0	S23
S23	XS3 paso 2: BCD(0)(0) ← bin(31)	cont = 31: S26 (paso 5 cumplido). Si no: d ← 0, S24.
S24	XS3 paso 3 sobre el dígito d: si BCD(d) > 4, BCD(d) ← BCD(d) + 3 (en el sumador/restador)	d ≠ 9: d ← d + 1, S24. d = 9: S25.
S25	XS3 paso 4: BCD ← BCD << 1 (entra 0); bin ← bin << 1; cont ← cont + 1	S23
S26	final = 1; display ← BCD; signo del display ← neg; dira ← 0	inicio = 1: pag ← sw_mat, S1. sw_piv ≠ sel_ant: S21. Si no: S26.

8. Señales del procesador

8.1 Señales de estado (CD → UC)

Señal de estado	Cómo se obtiene	Se pregunta en
inicio	Botón BTNC sincronizado con dos flip-flops	S0, S26
M(k,k) = 0	Comparador de dout_ram con 0	S4
M(i,k) = 0	Comparador de dout_ram con 0	S5
i = n	Comparador de 3 bits	S2, S5, S18
j = n	Comparador de 3 bits	S2, S9, S18
k = n − 1	Comparador de 3 bits	S19
signo(t3)	Bit t3(49)	S16
R − |p| ≥ 0	Bit 49 de la salida del sumador/restador en 0	S17
sq	Registro de signo del cociente	S18
signo(piv)	Bit piv(31)	S19
signo	Registro con la paridad de intercambios de filas	S20
signo(val)	Bit val(31)	S21
cont = 31	Comparador de 5 bits	S23
BCD(d) > 4	Comparador de 4 bits sobre el dígito elegido por d	S24
d = 9	Comparador de 4 bits	S24
sw_piv ≠ sel_ant	Comparador de 3 bits (switches sincronizados)	S26

8.2 Señales de control por estado (UC → CD)

Además de cargar los registros que aparecen en cada caja, el estado controla la RAM, el multiplicador y el sumador/restador. Los estados que no aparecen no usan estas unidades.

Estado	Dirección RAM (fila, col)	Escribe en RAM	Sumador/restador	Multiplicador
S2	(i, j)	ROM con extensión de signo	—	—
S4	(k, k)	—	—	—
S5	(i, k)	—	—	—
S6	(k, j)	—	—	—
S7	(i, j)	—	—	—
S8	(k, j)	tmp2	—	—
S9	(i, j)	tmp1	—	—
S11	(i, j)	—	—	carga mul_a, mul_b
S12	(i, k)	—	—	t1 ← producto; carga mul_a
S13	(k, j)	—	—	carga mul_b
S14	—	—	—	t2 ← producto
S15	—	—	t1 − t2	—
S16	—	—	0 − t3	—
S17	—	—	R − |p|	—
S18	(i, j)	Q, o la salida del restador si sq = 1	0 − Q	—

Estado	Dirección RAM (fila, col)	Escribe en RAM	Sumador/restador	Multiplicador
S19	—	—	0 − piv	—
S20	(n, n)	—	0 − M(n,n)	—
S21	—	—	0 − val	—
S24	—	—	BCD(d) + 3 (suma)	—

En el VHDL estas señales están en la sección «CAMINO DE DATOS» como asignaciones concurrentes que dependen de `st`: `fila_ram`, `col_ram`, `we_ram`, `din_ram`, `sr_a`, `sr_b` y `sr_op`. Si en el informe implementan la UC con ROM de entrada, registro y ROM de salida, como en el ejemplo del contador de unos, esta tabla es la base de la ROM de salida (palabra de control).

8.3 Registros internos y tamaños

Registro	Bits	Uso
st	27 estados	Estado del ASM
pag, dira	3, 5	Página y dirección de la ROM
n, i, j, k	3 c/u	Tamaño de la matriz y contadores de fila, columna y paso
piv, p, |p|	32 c/u	Pivote actual, pivote anterior (divisor) y su magnitud
tmp1, tmp2	32 c/u	Intercambio de filas
mul_a, mul_b	25 c/u	Registros de entrada del multiplicador
t1, t2, t3	50 c/u	Productos y numerador de la fórmula (1)
R, Q, sq	50, 32, 1	Residuo, cociente y signo del cociente
signo	1	Paridad de intercambios de filas
det	32	Determinante
banco	4 × 32	Pivotes 1 a 4
bin, BCD	32, 10 × 4	Registros binario y decimal del XS3
cont, d	5, 4	Bits procesados y dígito revisado en el XS3
neg, sel_ant	1, 3	Signo del valor convertido y selección ya convertida
display, signo display	10 × 4, 1	Valor que se ve en los 8 displays
refresco	18	Contador de barrido del display (2,6 ms por barrido completo)

Por qué estos anchos

Por la identidad de Sylvester, cada valor guardado en la RAM es el determinante de una submatriz m×m de A. La desigualdad de Hadamard lo acota: $|\det| \le (32\cdot\sqrt{m})^m$ cuando $|a(i,j)| \le 32$.

- Operandos del multiplicador: en el paso k son submatrices de k×k con k ≤ 4, así que $|x| < 2^{24} =$ 16 777 216. Caben en 25 bits con signo. (La cota no se alcanza porque +32 no existe en 6 bits.)
- Determinante 5×5: $|\det| \le 1{,}88 \times 10^9 < 2^{31}$. Por eso la RAM, det, Q y bin son de 32 bits.
- Numerador: t3 = p · a(k), con |p| ≤ 170 268 (submatriz 3×3), así que $|t3| \le 3{,}2 \times 10^{14} < 2^{49}$. t1, t2, t3, R y el sumador/restador son de 50 bits.
- BCD: 32 bits binarios necesitan 10 dígitos decimales. El display muestra los 7 de abajo; si los dígitos 7 a 9 no son cero, LD1 (desborde) se enciende.

9. Camino de datos

El camino de datos se organiza alrededor de dos buses: la salida de la RAM (`dout_ram`) y la salida del sumador/restador (`sr_s`). Los multiplexores de la figura los controla el estado del ASM (sección 8.2).

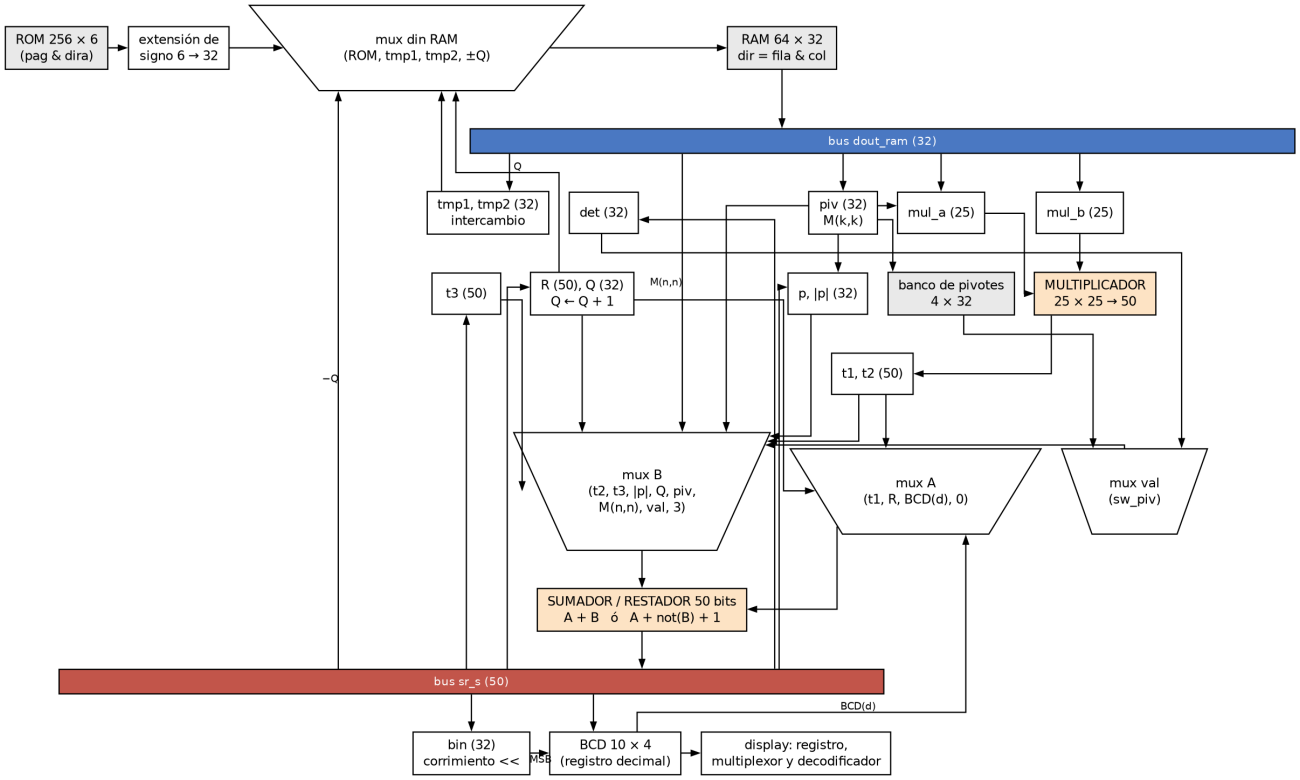


Figura 10. Diagrama de bloques simplificado del camino de datos.

Unidad	Detalle	Estados
Multiplicador	Con signo, 25 × 25 → 50 bits. `mul_p <= signed(mul_a) * signed(mul_b);`	S12, S14
Sumador/restador	50 bits. `sr_s <= sr_a + sr_bx + sr_op;` con sr_bx = not sr_b en resta: un solo sumador con acarreo de entrada, como el bloque «+/−» de las notas de clase.	S15 a S21, S24
División por restas sucesivas	Registros R, Q y |p|, incrementador Q + 1 y el bit de signo de la resta R − |p| como señal de estado. Usa el sumador/restador.	S16 a S18
ROM	256 × 6 bits, lectura combinacional: `a <= ROM(conv_integer(pag & dira));`	S1, S2
RAM	64 × 32 bits, escritura síncrona y lectura asíncrona; dirección fila & columna	S2 a S20
Banco de pivotes	4 × 32 bits; escritura en S10, lectura con sw_piv	S3, S10, S21
Contadores	dira, i, j, k, Q, cont, d y refresco	varios
Comparadores	= 0, i = n, j = n, k = n − 1, cont = 31, d = 9, BCD(d) > 4, sw_piv ≠ sel_ant	varios
Display	Registro de 10 dígitos, multiplexor de 8 a 1 y decodificador de 7 segmentos	S26 y siempre

La síntesis genérica con GHDL (`ghdl --synth`) confirma que el diseño tiene un solo multiplicador (25 × 25) y un solo sumador de 50 bits, y que infiere la ROM (256 × 6) y la RAM (64 × 32) como

memorias.

10. Tiempo de ejecución

- Carga de la matriz (S0 a S3): $n^2 + 3$ ciclos.
- Paso k: S4, S10 y S19 (3 ciclos) más, por cada uno de los $(n - k)^2$ elementos, 8 ciclos fijos (S11 a S16 y S18) y |cociente| + 1 ciclos de división en S17.
- Intercambio de filas: hasta $n - k$ ciclos de búsqueda, 4 ciclos por columna y una nueva lectura del pivote.
- XS3: 375 ciclos (S21 y S22, 31 pasadas de 12 ciclos por S23, S24 × 10 y S25, y la última pasada por S23).

La división por restas sucesivas domina el tiempo cuando los valores son grandes, porque tarda tantos ciclos como vale el cociente. En la página 7 (5×5 con valores cercanos a ±32) el cálculo tarda 19 013 425 ciclos, 0,19 s a 100 MHz. En el peor caso teórico el último cociente se acerca a $1{,}9 \times 10^9$, unos 19 s. Si el profesor acepta otra forma de división, una división con corrimiento y resta (restauración) haría lo mismo en 50 ciclos; aquí se dejó la de restas sucesivas porque es la que pide el enunciado.

11. Implementación en ISE 14.7 y la Nexys A7

11.1 Proyecto

- File → New Project. Top-level source type: HDL.
- Family: Artix7 · Device: XC7A100T · Package: CSG324 · Speed: −1 · Synthesis: XST · Simulator: ISim · Preferred language: VHDL. ISE solo soporta la Nexys A7-100T; la variante 50T exige Vivado.
- Add Source: src/det_bareiss.vhd y src/nexys_a7.ucf (Implementation), sim/tb_det_bareiss.vhd (Simulation).
- Synthesize → Implement Design → Generate Programming File. Revisen en el reporte de tiempos que se cumpla la restricción de 10 ns (TS_sys_clk).
- Programación: según el manual de la tarjeta, la Nexys A7 no funciona con Digilent Adept. Usen iMPACT de ISE o el Hardware Manager de Vivado (Lab Edition) con el archivo .bit.

11.2 Operación en la tarjeta

- Pulsar BTNU (rst).
- Elegir la matriz con SW15..SW13 (0 a 7, tabla 11.3) y dejar SW2..SW0 en 000.
- Pulsar BTNC (inicio). Cuando termina, LD0 (final) se enciende y el display muestra el signo en AN7 y 7 dígitos.
- Poner SW2..SW0 en 1 a $n - 1$ para ver cada pivote; volver a 000 para ver el determinante.
- Si LD1 (desborde) está encendido, el valor tiene más de 7 dígitos y el display muestra los 7 de abajo.
- Para calcular otra matriz: cambiar SW15..SW13 y pulsar BTNC de nuevo.

11.3 Matrices de prueba en la ROM

Pág.	n	Matriz A	det	Pivotes	Qué prueba
0	2	7 -3 -3 5	26	7	Caso básico 2×2

Pág.	n	Matriz A	det	Pivotes	Qué prueba
1	3	2 1 3 1 4 2 3 2 5	3	2, 7	Ejemplo del documento
2	4	0 2 1 3 2 1 0 1 1 0 3 2 3 1 2 4	-22	2, 4, 13	a11 = 0: intercambio en k = 1
3	5	4 -2 1 3 0 -2 6 2 -1 5 1 2 -7 4 2 3 -1 4 5 -3 0 5 2 -3 8	362	4, 20, -170, -639	5×5 general, pivotes negativos
4	3	-2 4 1 4 -8 3 1 3 5	50	-2, -10	Pivote negativo; intercambio en k = 2
5	4	1 2 3 0 2 4 1 5 3 1 -2 4 0 5 4 -3	150	1, -5, 25	Pivote cero en k = 2 (p ≠ 1)
6	5	0 0 0 0 0 0 3 1 2 4 0 1 5 -2 1 0 2 -2 6 3 0 4 1 3 7	0	—	Columna 1 en cero: singular, det = 0
7	5	30 27 24 -29 -31 27 -21 28 29 -24 24 28 2 -28 -28 -29 29 -28 -27 28 -31 -24 -28 28 -30	-17 121 828	30, -1359, 22146, 277564	Valores extremos; desborde del display

Para cambiar o agregar matrices, editen la lista MATRICES en sim/modelo_referencia.py y ejecútenlo: imprime el det y los pivotes esperados y las líneas de la ROM listas para pegar en det_bareiss.vhd. Cada página empieza en la dirección 32 × página.

11.4 Pines (src/nexys_a7.ucf)

Señal	Pines
clk	E3 (100 MHz)
rst / inicio	M18 (BTNU) / N17 (BTNC)
sw_piv(0..2)	J15 (SW0), L16 (SW1), M13 (SW2)
sw_mat(0..2)	U12 (SW13), U11 (SW14), V10 (SW15)
final / desborde	H17 (LD0) / K15 (LD1)
seg(0..6) = CA..CG, dp	T10, R10, K16, K13, P15, T11, L18; DP = H15
an(0..7)	J17, J18, T9, J14, P14, T14, K2, U13

12. Simulación

El testbench pulsa inicio para cada matriz, espera final = 1 y lee el display decodificando las señales an y seg, igual que lo haría el ojo. Después recorre los pivotes con sw_piv = 1 a n − 1. Con N_REF = 5 el barrido del display es rápido en simulación. El genérico MAX_MATRIZ del testbench vale 6 por defecto (matrices 0 a 6, unos 0,2 ms simulados); con 7 incluye la matriz extrema, que en ISim tarda bastante.

Pág.	Tamaño	Display esperado	Display simulado	LD1	Ciclos	Resultado
0	2x2	`' 0000026'`	`' 0000026'`	0	415	OK
1	3x3	`' 0000003'`	`' 0000003'`	0	442	OK
2	4x4	`'-0000022'`	`'-0000022'`	0	627	OK
3	5x5	`' 0000362'`	`' 0000362'`	0	4 203	OK
4	3x3	`' 0000050'`	`' 0000050'`	0	520	OK
5	4x4	`' 0000150'`	`' 0000150'`	0	807	OK
6	5x5	`' 0000000'`	`' 0000000'`	0	403	OK
7	5x5	`'-7121828'`	`'-7121828'`	1	19 013 425	OK

Todos los pivotes también coinciden (ver sim/resultado_ghdl.txt). Para comprobar que el testbench detecta errores, se cambió en una copia del diseño el «+3» del XS3 por «+2»: el testbench reportó 7 errores en las matrices 0 a 3. Los ciclos se cuentan desde que se suelta inicio hasta final = 1.

13. Limitaciones y verificación pendiente

- **No se ejecutó ISE.** El diseño se verificó con GHDL 4.1 (simulación con `-fsynopsys -fexplicit` y síntesis genérica), no con XST ni ISim. Hay que sintetizar en ISE 14.7 y revisar el reporte de tiempos a 100 MHz. Si la restricción de 10 ns no se cumpliera, la salida es generar un reloj de 50 MHz con el Clocking Wizard y poner PERIOD = 20 ns.
- **Display de 7 dígitos.** Con signo en AN7 caben valores hasta ±9 999 999; para valores mayores se enciende LD1 y se ven los 7 dígitos de abajo.
- **Tiempo de la división.** Proporcional al cociente (sección 10).
- **Modelo de RAM.** Se usó lectura asíncrona. Si el modelo del aula tiene lectura síncrona, cada lectura (S4, S5, S6, S7, S11, S12, S13, S20) necesita un estado de espera antes de usar el dato.
- **Valor de n.** El diseño supone 2 ≤ n ≤ 5 en la ROM y no lo valida.
- **ROM con 8 páginas.** Se agregó la selección de página con SW15..SW13 para mostrar varios tamaños sin volver a sintetizar. Si el profesor quiere una sola matriz, usen la página 0 con los switches en 0.

Anexo A. Código VHDL completo (src/det_bareiss.vhd)

```
----------------------------------------------------------------------------------
-- Universidad Distrital Francisco Jose de Caldas
-- Analisis y Diseno de Microprocesadores - Taller 5
--
-- Microprocesador de proposito especifico descrito con un diagrama ASM:
--   * Determinante de una matriz simetrica de 2x2 a 5x5 por el algoritmo de
--     Bareiss (eliminacion libre de fracciones).
--   * Division por restas sucesivas.
--   * Conversion binario -> BCD con el algoritmo XS3 (sumar 3 y desplazar),
--     descrita paso a paso dentro del mismo ASM.
--   * Banco de registros con los pivotes; se eligen con los switches.
--
-- Herramienta : Xilinx ISE 14.7 (XST / ISim)
-- Tarjeta     : Digilent Nexys A7-100T (XC7A100T-1CSG324C), reloj de 100 MHz
--
-- Todo el diseno esta descrito con senales (no se usan variables).
-- El numero de cada estado coincide con el diagrama ASM del documento.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity det_bareiss is
    generic ( N_REF : integer := 18 );  -- bits del contador de refresco del display
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;                       -- BTNC
           sw_mat   : in  STD_LOGIC_VECTOR (2 downto 0);  -- SW15..SW13: matriz (pagina de la ROM)
           sw_piv   : in  STD_LOGIC_VECTOR (2 downto 0);  -- SW2..SW0 : 0 = det, 1..4 = pivote k
           final    : out STD_LOGIC;                       -- LD0
           desborde : out STD_LOGIC;                       -- LD1: |valor| > 9 999 999
           an       : out STD_LOGIC_VECTOR (7 downto 0);  -- anodos (activos en bajo)
           seg      : out STD_LOGIC_VECTOR (6 downto 0);  -- seg(0)=CA ... seg(6)=CG (activos en bajo)
           dp       : out STD_LOGIC);
end det_bareiss;

architecture Behavioral of det_bareiss is

----------------------------------------------------------------------------------
-- UNIDAD DE CONTROL: estados del ASM
----------------------------------------------------------------------------------
-- s0        espera de inicio
-- s1..s2    lectura de n y copia de la matriz ROM -> RAM
-- s3        inicializacion de Bareiss
-- s4..s10   pivote: lectura, busqueda de fila, intercambio, banco de pivotes
-- s11..s18  actualizacion de a(i,j): multiplicaciones, resta y division
-- s19       actualizacion del pivote anterior p y avance de k
-- s20       det = (+/-) a(n,n)
-- s21       seleccion del valor a mostrar (det o pivote)
-- s22..s25  algoritmo XS3 (pasos 1 a 5)
-- s26       final = 1, valor en el display
type estados_asm is (s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10,
                     s11, s12, s13, s14, s15, s16, s17, s18, s19, s20,
                     s21, s22, s23, s24, s25, s26);
signal st : estados_asm;

----------------------------------------------------------------------------------
-- MEMORIA ROM: 8 paginas de 32 palabras de 6 bits (complemento a 2).
-- En cada pagina: posicion 00h = n, luego los a(i,j) por columnas
-- (a11, a21, ..., an1, a12, ...), como en la Figura 1 del taller.
----------------------------------------------------------------------------------
type rom_t is array (0 to 255) of std_logic_vector(5 downto 0);
constant ROM : rom_t := (
      -- pagina 0: 2x2, det = 26
        0 => conv_std_logic_vector(  2,6),  -- n = 2
        1 => conv_std_logic_vector(  7,6),  -- a11
        2 => conv_std_logic_vector( -3,6),  -- a21
        3 => conv_std_logic_vector( -3,6),  -- a12
        4 => conv_std_logic_vector(  5,6),  -- a22
      -- pagina 1: 3x3, det = 3
       32 => conv_std_logic_vector(  3,6),  -- n = 3
       33 => conv_std_logic_vector(  2,6),  -- a11
       34 => conv_std_logic_vector(  1,6),  -- a21
       35 => conv_std_logic_vector(  3,6),  -- a31
       36 => conv_std_logic_vector(  1,6),  -- a12
       37 => conv_std_logic_vector(  4,6),  -- a22
       38 => conv_std_logic_vector(  2,6),  -- a32
       39 => conv_std_logic_vector(  3,6),  -- a13
       40 => conv_std_logic_vector(  2,6),  -- a23
       41 => conv_std_logic_vector(  5,6),  -- a33
      -- pagina 2: 4x4 con a11 = 0 (intercambio de filas en k = 1), det = -22
       64 => conv_std_logic_vector(  4,6),  -- n = 4
       65 => conv_std_logic_vector(  0,6),  -- a11
       66 => conv_std_logic_vector(  2,6),  -- a21
       67 => conv_std_logic_vector(  1,6),  -- a31
```

```
        68 => conv_std_logic_vector(  3,6),  -- a41
        69 => conv_std_logic_vector(  2,6),  -- a12
        70 => conv_std_logic_vector(  1,6),  -- a22
        71 => conv_std_logic_vector(  0,6),  -- a32
        72 => conv_std_logic_vector(  1,6),  -- a42
        73 => conv_std_logic_vector(  1,6),  -- a13
        74 => conv_std_logic_vector(  0,6),  -- a23
        75 => conv_std_logic_vector(  3,6),  -- a33
        76 => conv_std_logic_vector(  2,6),  -- a43
        77 => conv_std_logic_vector(  3,6),  -- a14
        78 => conv_std_logic_vector(  1,6),  -- a24
        79 => conv_std_logic_vector(  2,6),  -- a34
        80 => conv_std_logic_vector(  4,6),  -- a44
       -- pagina 3: 5x5 general, det = 362
        96 => conv_std_logic_vector(  5,6),  -- n = 5
        97 => conv_std_logic_vector(  4,6),  -- a11
        98 => conv_std_logic_vector( -2,6),  -- a21
        99 => conv_std_logic_vector(  1,6),  -- a31
       100 => conv_std_logic_vector(  3,6),  -- a41
       101 => conv_std_logic_vector(  0,6),  -- a51
       102 => conv_std_logic_vector( -2,6),  -- a12
       103 => conv_std_logic_vector(  6,6),  -- a22
       104 => conv_std_logic_vector(  2,6),  -- a32
       105 => conv_std_logic_vector( -1,6),  -- a42
       106 => conv_std_logic_vector(  5,6),  -- a52
       107 => conv_std_logic_vector(  1,6),  -- a13
       108 => conv_std_logic_vector(  2,6),  -- a23
       109 => conv_std_logic_vector( -7,6),  -- a33
       110 => conv_std_logic_vector(  4,6),  -- a43
       111 => conv_std_logic_vector(  2,6),  -- a53
       112 => conv_std_logic_vector(  3,6),  -- a14
       113 => conv_std_logic_vector( -1,6),  -- a24
       114 => conv_std_logic_vector(  4,6),  -- a34
       115 => conv_std_logic_vector(  5,6),  -- a44
       116 => conv_std_logic_vector( -3,6),  -- a54
       117 => conv_std_logic_vector(  0,6),  -- a15
       118 => conv_std_logic_vector(  5,6),  -- a25
       119 => conv_std_logic_vector(  2,6),  -- a35
       120 => conv_std_logic_vector( -3,6),  -- a45
       121 => conv_std_logic_vector(  8,6),  -- a55
       -- pagina 4: 3x3 con pivote negativo e intercambio en k = 2, det = 50
       128 => conv_std_logic_vector(  3,6),  -- n = 3
       129 => conv_std_logic_vector( -2,6),  -- a11
       130 => conv_std_logic_vector(  4,6),  -- a21
       131 => conv_std_logic_vector(  1,6),  -- a31
       132 => conv_std_logic_vector(  4,6),  -- a12
       133 => conv_std_logic_vector( -8,6),  -- a22
       134 => conv_std_logic_vector(  3,6),  -- a32
       135 => conv_std_logic_vector(  1,6),  -- a13
       136 => conv_std_logic_vector(  3,6),  -- a23
       137 => conv_std_logic_vector(  5,6),  -- a33
       -- pagina 5: 4x4 con pivote cero en k = 2, det = 150
       160 => conv_std_logic_vector(  4,6),  -- n = 4
       161 => conv_std_logic_vector(  1,6),  -- a11
       162 => conv_std_logic_vector(  2,6),  -- a21
       163 => conv_std_logic_vector(  3,6),  -- a31
       164 => conv_std_logic_vector(  0,6),  -- a41
       165 => conv_std_logic_vector(  2,6),  -- a12
       166 => conv_std_logic_vector(  4,6),  -- a22
       167 => conv_std_logic_vector(  1,6),  -- a32
       168 => conv_std_logic_vector(  5,6),  -- a42
       169 => conv_std_logic_vector(  3,6),  -- a13
       170 => conv_std_logic_vector(  1,6),  -- a23
       171 => conv_std_logic_vector( -2,6),  -- a33
       172 => conv_std_logic_vector(  4,6),  -- a43
       173 => conv_std_logic_vector(  0,6),  -- a14
       174 => conv_std_logic_vector(  5,6),  -- a24
       175 => conv_std_logic_vector(  4,6),  -- a34
       176 => conv_std_logic_vector( -3,6),  -- a44
       -- pagina 6: 5x5 singular (columna 1 en cero), det = 0
       192 => conv_std_logic_vector(  5,6),  -- n = 5
       193 => conv_std_logic_vector(  0,6),  -- a11
       194 => conv_std_logic_vector(  0,6),  -- a21
       195 => conv_std_logic_vector(  0,6),  -- a31
       196 => conv_std_logic_vector(  0,6),  -- a41
       197 => conv_std_logic_vector(  0,6),  -- a51
       198 => conv_std_logic_vector(  0,6),  -- a12
       199 => conv_std_logic_vector(  3,6),  -- a22
       200 => conv_std_logic_vector(  1,6),  -- a32
       201 => conv_std_logic_vector(  2,6),  -- a42
       202 => conv_std_logic_vector(  4,6),  -- a52
       203 => conv_std_logic_vector(  0,6),  -- a13
       204 => conv_std_logic_vector(  1,6),  -- a23
       205 => conv_std_logic_vector(  5,6),  -- a33
       206 => conv_std_logic_vector( -2,6),  -- a43
       207 => conv_std_logic_vector(  1,6),  -- a53
       208 => conv_std_logic_vector(  0,6),  -- a14
       209 => conv_std_logic_vector(  2,6),  -- a24
```

```
174       210 => conv_std_logic_vector( -2,6),  -- a34
175       211 => conv_std_logic_vector(  6,6),  -- a44
176       212 => conv_std_logic_vector(  3,6),  -- a54
177       213 => conv_std_logic_vector(  0,6),  -- a15
178       214 => conv_std_logic_vector(  4,6),  -- a25
179       215 => conv_std_logic_vector(  1,6),  -- a35
180       216 => conv_std_logic_vector(  3,6),  -- a45
181       217 => conv_std_logic_vector(  7,6),  -- a55
182       -- pagina 7: 5x5 con valores extremos, det = -17 121 828 (desborde del display)
183       224 => conv_std_logic_vector(  5,6),  -- n = 5
184       225 => conv_std_logic_vector( 30,6),  -- a11
185       226 => conv_std_logic_vector( 27,6),  -- a21
186       227 => conv_std_logic_vector( 24,6),  -- a31
187       228 => conv_std_logic_vector(-29,6),  -- a41
188       229 => conv_std_logic_vector(-31,6),  -- a51
189       230 => conv_std_logic_vector( 27,6),  -- a12
190       231 => conv_std_logic_vector(-21,6),  -- a22
191       232 => conv_std_logic_vector( 28,6),  -- a32
192       233 => conv_std_logic_vector( 29,6),  -- a42
193       234 => conv_std_logic_vector(-24,6),  -- a52
194       235 => conv_std_logic_vector( 24,6),  -- a13
195       236 => conv_std_logic_vector( 28,6),  -- a23
196       237 => conv_std_logic_vector(  2,6),  -- a33
197       238 => conv_std_logic_vector(-28,6),  -- a43
198       239 => conv_std_logic_vector(-28,6),  -- a53
199       240 => conv_std_logic_vector(-29,6),  -- a14
200       241 => conv_std_logic_vector( 29,6),  -- a24
201       242 => conv_std_logic_vector(-28,6),  -- a34
202       243 => conv_std_logic_vector(-27,6),  -- a44
203       244 => conv_std_logic_vector( 28,6),  -- a54
204       245 => conv_std_logic_vector(-31,6),  -- a15
205       246 => conv_std_logic_vector(-24,6),  -- a25
206       247 => conv_std_logic_vector(-28,6),  -- a35
207       248 => conv_std_logic_vector( 28,6),  -- a45
208       249 => conv_std_logic_vector(-30,6),  -- a55
209       others => (others => '0'));
210
211   signal pag     : std_logic_vector(2 downto 0);   -- pagina elegida al iniciar
212   signal dira    : std_logic_vector(4 downto 0);   -- linea de direccion de la ROM
213   signal dir_rom : std_logic_vector(7 downto 0);
214   signal a       : std_logic_vector(5 downto 0);   -- bus de datos de la ROM
215
216   ------------------------------------------------------------------------------
217   -- MEMORIA RAM de trabajo: 64 x 32 bits. Direccion = fila(3) & columna(3),
218   -- asi no se necesita multiplicar para calcular la direccion de a(i,j).
219   -- Escritura sincrona, lectura asincrona.
220   ------------------------------------------------------------------------------
221   type ram_t is array (0 to 63) of std_logic_vector(31 downto 0);
222   signal ram      : ram_t;
223   signal fila_ram : std_logic_vector(2 downto 0);
224   signal col_ram  : std_logic_vector(2 downto 0);
225   signal dir_ram  : std_logic_vector(5 downto 0);
226   signal din_ram  : std_logic_vector(31 downto 0);
227   signal dout_ram : std_logic_vector(31 downto 0);
228   signal we_ram   : std_logic;
229
230   ------------------------------------------------------------------------------
231   -- BANCO DE REGISTROS DE PIVOTES: pivote(k) = a(k,k) usado en el paso k
232   ------------------------------------------------------------------------------
233   type banco_t is array (1 to 4) of std_logic_vector(31 downto 0);
234   signal banco : banco_t;
235
236   ------------------------------------------------------------------------------
237   -- Registros del algoritmo de Bareiss
238   ------------------------------------------------------------------------------
239   signal n, i, j, k : std_logic_vector(2 downto 0);  -- tamano y contadores
240   signal piv   : std_logic_vector(31 downto 0);      -- pivote actual a(k,k)
241   signal p     : std_logic_vector(31 downto 0);      -- pivote anterior (divisor)
242   signal pabs  : std_logic_vector(31 downto 0);      -- |p|
243   signal signo : std_logic;                          -- '1' = numero impar de intercambios
244   signal tmp1, tmp2 : std_logic_vector(31 downto 0); -- intercambio de filas
245   signal t1, t2, t3 : std_logic_vector(49 downto 0); -- productos y numerador
246   signal det   : std_logic_vector(31 downto 0);      -- determinante
247
248   -- Multiplicador (unidad funcional 1)
249   signal mul_a, mul_b : std_logic_vector(24 downto 0);
250   signal mul_p        : std_logic_vector(49 downto 0);
251
252   -- Sumador/restador (unidad funcional 2): sr_s = sr_a + sr_b  o  sr_a - sr_b
253   signal sr_a, sr_b, sr_bx, sr_s : std_logic_vector(49 downto 0);
254   signal sr_op : std_logic;                          -- '0' suma, '1' resta
255
256   -- Bloque de division por restas sucesivas (unidad funcional 3)
257   signal R  : std_logic_vector(49 downto 0);         -- residuo
258   signal Q  : std_logic_vector(31 downto 0);         -- cociente
259   signal sq : std_logic;                             -- signo del cociente
260
261   ------------------------------------------------------------------------------
```

```
-- Conversion XS3 (binario -> BCD)
--------------------------------------------------------------------------------
type bcd_t is array (0 to 9) of std_logic_vector(3 downto 0);
signal bcd     : bcd_t;                             -- registro decimal (10 digitos)
signal bin     : std_logic_vector(31 downto 0);     -- registro binario
signal cont    : std_logic_vector(4 downto 0);      -- bits procesados (0..31)
signal d       : std_logic_vector(3 downto 0);      -- digito revisado (0..9)
signal dig_d   : std_logic_vector(3 downto 0);      -- bcd(d)
signal neg     : std_logic;                         -- signo del valor convertido
signal val_sel : std_logic_vector(31 downto 0);     -- det o pivote elegido
signal sel_ant : std_logic_vector(2 downto 0);      -- seleccion ya convertida

--------------------------------------------------------------------------------
-- Visualizacion
--------------------------------------------------------------------------------
signal disp     : bcd_t;                            -- BCD que se muestra
signal disp_neg : std_logic;
signal refresco : std_logic_vector(N_REF-1 downto 0);
signal dsel     : std_logic_vector(2 downto 0);     -- digito encendido
signal car      : std_logic_vector(3 downto 0);     -- 0-9, A = '-', F = apagado

-- Sincronizacion de entradas externas
signal ini_r1, ini_s : std_logic;
signal piv_r1, piv_s : std_logic_vector(2 downto 0);
signal mat_r1, mat_s : std_logic_vector(2 downto 0);

begin

--------------------------------------------------------------------------------
-- CAMINO DE DATOS
--------------------------------------------------------------------------------

-- ROM
dir_rom <= pag & dira;
a <= ROM(conv_integer(dir_rom));

-- RAM: la direccion, el dato y la escritura dependen del estado
fila_ram <= k when (st = s4 or st = s6 or st = s8 or st = s13) else
            n when st = s20 else
            i;
col_ram  <= k when (st = s4 or st = s5 or st = s12) else
            n when st = s20 else
            j;
dir_ram  <= fila_ram & col_ram;

we_ram   <= '1' when (st = s2 or st = s8 or st = s9 or st = s18) else '0';

din_ram  <= sxt(a, 32)        when st = s2 else         -- copia de la ROM
            tmp2              when st = s8 else         -- intercambio
            tmp1              when st = s9 else         -- intercambio
            sr_s(31 downto 0) when sq = '1' else        -- s18: -Q
            Q;                                          -- s18: +Q

process (clk)
begin
    if clk'event and clk = '1' then
        if we_ram = '1' then
            ram(conv_integer(dir_ram)) <= din_ram;
        end if;
    end if;
end process;

dout_ram <= ram(conv_integer(dir_ram));

-- Multiplicador con signo 25 x 25 -> 50 bits
mul_p <= signed(mul_a) * signed(mul_b);

-- Sumador/restador de 50 bits: A + B  o  A + not(B) + 1 = A - B
sr_a <= t1             when st = s15 else               -- t1 - t2
        R              when st = s17 else               -- R - |p|
        ext(dig_d, 50) when st = s24 else               -- digito + 3
        (others => '0');                                -- 0 - x (negacion)
sr_b <= t2                            when st = s15 else
        t3                            when st = s16 else -- -t3
        ext(pabs, 50)                 when st = s17 else -- |p|
        ext(Q, 50)                    when st = s18 else -- -Q
        sxt(piv, 50)                  when st = s19 else -- -piv
        sxt(dout_ram, 50)             when st = s20 else -- -a(n,n)
        sxt(val_sel, 50)              when st = s21 else -- -valor
        conv_std_logic_vector(3, 50) when st = s24 else -- +3 (XS3)
        (others => '0');
sr_op <= '0' when st = s24 else '1';
sr_bx <= not sr_b when sr_op = '1' else sr_b;
sr_s  <= sr_a + sr_bx + sr_op;

-- Valor a mostrar: det (sw_piv = 0) o pivote(k) (sw_piv = 1..4)
val_sel <= det                         when piv_s = "000" else
           banco(conv_integer(piv_s)) when piv_s < 5 else
```

```
           (others => '0');

dig_d <= bcd(conv_integer(d));

-------------------------------------------------------------------------------
-- UNIDAD DE CONTROL: diagrama ASM
-------------------------------------------------------------------------------
final <= '1' when st = s26 else '0';

process (clk, rst)
begin

if rst = '1' then
    st       <= s0;
    pag      <= (others => '0');
    dira     <= (others => '0');
    det      <= (others => '0');
    banco    <= (others => (others => '0'));
    sel_ant  <= (others => '0');
    d        <= (others => '0');
    cont     <= (others => '0');
    disp     <= (others => (others => '0'));
    disp_neg <= '0';

elsif clk'event and clk = '1' then

    case st is

        ---------------------------------------------------------------
        -- Espera y carga de la matriz
        ---------------------------------------------------------------
        when s0 =>                              -- final = 0, dira = 0
            dira <= (others => '0');
            if ini_s = '0' then
                st <= s0;
            else
                pag <= mat_s;
                st  <= s1;
            end if;

        when s1 =>                              -- n = ROM(00h)
            n    <= a(2 downto 0);
            dira <= "00001";
            i    <= "001";
            j    <= "001";
            st   <= s2;

        when s2 =>                              -- M(i,j) = ROM(dira), por columnas
            dira <= dira + 1;
            if i = n then
                i <= "001";
                if j = n then
                    st <= s3;
                else
                    j  <= j + 1;
                    st <= s2;
                end if;
            else
                i  <= i + 1;
                st <= s2;
            end if;

        when s3 =>                              -- inicializacion de Bareiss
            k     <= "001";
            p     <= conv_std_logic_vector(1, 32);   -- a(0,0) = 1
            pabs  <= conv_std_logic_vector(1, 32);
            signo <= '0';
            banco <= (others => (others => '0'));
            st    <= s4;

        ---------------------------------------------------------------
        -- Pivote del paso k
        ---------------------------------------------------------------
        when s4 =>                              -- piv = M(k,k)
            piv <= dout_ram;
            if dout_ram = 0 then
                i  <= k + 1;
                st <= s5;
            else
                st <= s10;
            end if;

        when s5 =>                              -- busca fila i > k con M(i,k) /= 0
            if dout_ram /= 0 then
                j     <= k;
                signo <= not signo;             -- el intercambio cambia el signo
                st    <= s6;
            elsif i = n then
```

```
438                 det <= (others => '0');         -- no hay fila: matriz singular
439                 st  <= s21;
440             else
441                 i  <= i + 1;
442                 st <= s5;
443             end if;
444
445         when s6 =>                              -- tmp1 = M(k,j)
446             tmp1 <= dout_ram;
447             st   <= s7;
448
449         when s7 =>                              -- tmp2 = M(i,j)
450             tmp2 <= dout_ram;
451             st   <= s8;
452
453         when s8 =>                              -- M(k,j) = tmp2
454             st <= s9;
455
456         when s9 =>                              -- M(i,j) = tmp1
457             if j = n then
458                 st <= s4;                       -- vuelve a leer el pivote
459             else
460                 j  <= j + 1;
461                 st <= s6;
462             end if;
463
464         when s10 =>                             -- banco(k) = piv ; i = j = k+1
465             banco(conv_integer(k)) <= piv;
466             i  <= k + 1;
467             j  <= k + 1;
468             st <= s11;
469
470         ------------------------------------------------------------------
471         -- a(i,j) = ( a(k,k)*a(i,j) - a(i,k)*a(k,j) ) / p
472         ------------------------------------------------------------------
473         when s11 =>                             -- mul_a = a(k,k), mul_b = M(i,j)
474             mul_a <= piv(24 downto 0);
475             mul_b <= dout_ram(24 downto 0);
476             st    <= s12;
477
478         when s12 =>                             -- t1 = a(k,k)*a(i,j), mul_a = M(i,k)
479             t1    <= mul_p;
480             mul_a <= dout_ram(24 downto 0);
481             st    <= s13;
482
483         when s13 =>                             -- mul_b = M(k,j)
484             mul_b <= dout_ram(24 downto 0);
485             st    <= s14;
486
487         when s14 =>                             -- t2 = a(i,k)*a(k,j)
488             t2 <= mul_p;
489             st <= s15;
490
491         when s15 =>                             -- t3 = t1 - t2
492             t3 <= sr_s;
493             st <= s16;
494
495         when s16 =>                             -- R = |t3|, Q = 0, signo del cociente
496             Q  <= (others => '0');
497             sq <= t3(49) xor p(31);
498             if t3(49) = '1' then
499                 R <= sr_s;                      -- 0 - t3
500             else
501                 R <= t3;
502             end if;
503             st <= s17;
504
505         when s17 =>                             -- division por restas sucesivas
506             if sr_s(49) = '0' then              -- R - |p| >= 0
507                 R  <= sr_s;
508                 Q  <= Q + 1;
509                 st <= s17;
510             else
511                 st <= s18;
512             end if;
513
514         when s18 =>                             -- M(i,j) = +Q o -Q ; avanza j, i
515             if j = n then
516                 if i = n then
517                     st <= s19;
518                 else
519                     i  <= i + 1;
520                     j  <= k + 1;
521                     st <= s11;
522                 end if;
523             else
524                 j  <= j + 1;
525                 st <= s11;
```

```
526             end if;
527
528         when s19 =>                             -- p = a(k,k), |p|, k = k + 1
529             p <= piv;
530             if piv(31) = '1' then
531                 pabs <= sr_s(31 downto 0);      -- 0 - piv
532             else
533                 pabs <= piv;
534             end if;
535             k <= k + 1;
536             if k = n - 1 then
537                 st <= s20;
538             else
539                 st <= s4;
540             end if;
541
542         when s20 =>                             -- det = (+/-) M(n,n)
543             if signo = '1' then
544                 det <= sr_s(31 downto 0);       -- 0 - M(n,n)
545             else
546                 det <= dout_ram;
547             end if;
548             st <= s21;
549
550         ------------------------------------------------------------------
551         -- Valor a mostrar y conversion XS3
552         ------------------------------------------------------------------
553         when s21 =>                             -- bin = |valor|, neg = signo
554             sel_ant <= piv_s;
555             neg     <= val_sel(31);
556             if val_sel(31) = '1' then
557                 bin <= sr_s(31 downto 0);       -- 0 - valor
558             else
559                 bin <= val_sel;
560             end if;
561             st <= s22;
562
563         when s22 =>                             -- XS3 paso 1: registro decimal = 0
564             bcd  <= (others => (others => '0'));
565             cont <= (others => '0');
566             st   <= s23;
567
568         when s23 =>                             -- XS3 paso 2: LSB decimal = MSB binario
569             bcd(0)(0) <= bin(31);
570             if cont = 31 then
571                 st <= s26;                      -- paso 5: ultimo paso 2 hecho
572             else
573                 d  <= (others => '0');
574                 st <= s24;
575             end if;
576
577         when s24 =>                             -- XS3 paso 3: si digito(d) > 4, sumar 3
578             if dig_d > 4 then
579                 bcd(conv_integer(d)) <= sr_s(3 downto 0);   -- digito + 3
580             end if;
581             if d = 9 then
582                 st <= s25;
583             else
584                 d  <= d + 1;
585                 st <= s24;
586             end if;
587
588         when s25 =>                             -- XS3 paso 4: corrimiento a la izquierda
589             bcd(9) <= bcd(9)(2 downto 0) & bcd(8)(3);
590             bcd(8) <= bcd(8)(2 downto 0) & bcd(7)(3);
591             bcd(7) <= bcd(7)(2 downto 0) & bcd(6)(3);
592             bcd(6) <= bcd(6)(2 downto 0) & bcd(5)(3);
593             bcd(5) <= bcd(5)(2 downto 0) & bcd(4)(3);
594             bcd(4) <= bcd(4)(2 downto 0) & bcd(3)(3);
595             bcd(3) <= bcd(3)(2 downto 0) & bcd(2)(3);
596             bcd(2) <= bcd(2)(2 downto 0) & bcd(1)(3);
597             bcd(1) <= bcd(1)(2 downto 0) & bcd(0)(3);
598             bcd(0) <= bcd(0)(2 downto 0) & '0';
599             bin    <= bin(30 downto 0) & '0';
600             cont   <= cont + 1;
601             st     <= s23;
602
603         when s26 =>                             -- final = 1, display = BCD
604             disp     <= bcd;
605             disp_neg <= neg;
606             dira     <= (others => '0');
607             if ini_s = '1' then
608                 pag <= mat_s;
609                 st  <= s1;                      -- nuevo calculo
610             elsif piv_s /= sel_ant then
611                 st <= s21;                      -- cambio de switches: reconvertir
612             else
613                 st <= s26;
```

```
            end if;

    end case;
end if;
end process;

-------------------------------------------------------------------------------
-- Sincronizacion de botones y switches (dos flip-flops)
-------------------------------------------------------------------------------
process (clk, rst)
begin
    if rst = '1' then
        ini_r1 <= '0';  ini_s <= '0';
        piv_r1 <= (others => '0');  piv_s <= (others => '0');
        mat_r1 <= (others => '0');  mat_s <= (others => '0');
    elsif clk'event and clk = '1' then
        ini_r1 <= inicio;  ini_s <= ini_r1;
        piv_r1 <= sw_piv;  piv_s <= piv_r1;
        mat_r1 <= sw_mat;  mat_s <= mat_r1;
    end if;
end process;

-------------------------------------------------------------------------------
-- Display de 8 digitos multiplexado: AN7 = signo, AN6..AN0 = 7 digitos BCD
-------------------------------------------------------------------------------
process (clk, rst)
begin
    if rst = '1' then
        refresco <= (others => '0');
    elsif clk'event and clk = '1' then
        refresco <= refresco + 1;
    end if;
end process;

dsel <= refresco(N_REF-1 downto N_REF-3);

car <= "1010" when (dsel = "111" and disp_neg = '1') else   -- signo menos
       "1111" when dsel = "111" else                        -- apagado
       disp(conv_integer(dsel));

with dsel select
    an <= "11111110" when "000",
          "11111101" when "001",
          "11111011" when "010",
          "11110111" when "011",
          "11101111" when "100",
          "11011111" when "101",
          "10111111" when "110",
          "01111111" when others;

with car select                                 --  gfedcba
    seg <= "1000000" when "0000",               --  0
           "1111001" when "0001",               --  1
           "0100100" when "0010",               --  2
           "0110000" when "0011",               --  3
           "0011001" when "0100",               --  4
           "0010010" when "0101",               --  5
           "0000010" when "0110",               --  6
           "1111000" when "0111",               --  7
           "0000000" when "1000",               --  8
           "0010000" when "1001",               --  9
           "0111111" when "1010",               --  -
           "1111111" when others;               --  apagado

dp <= '1';

-- Los digitos 7, 8 y 9 del BCD no caben en el display
desborde <= '0' when (disp(7) = "0000" and disp(8) = "0000" and disp(9) = "0000") else '1';

end Behavioral;
```